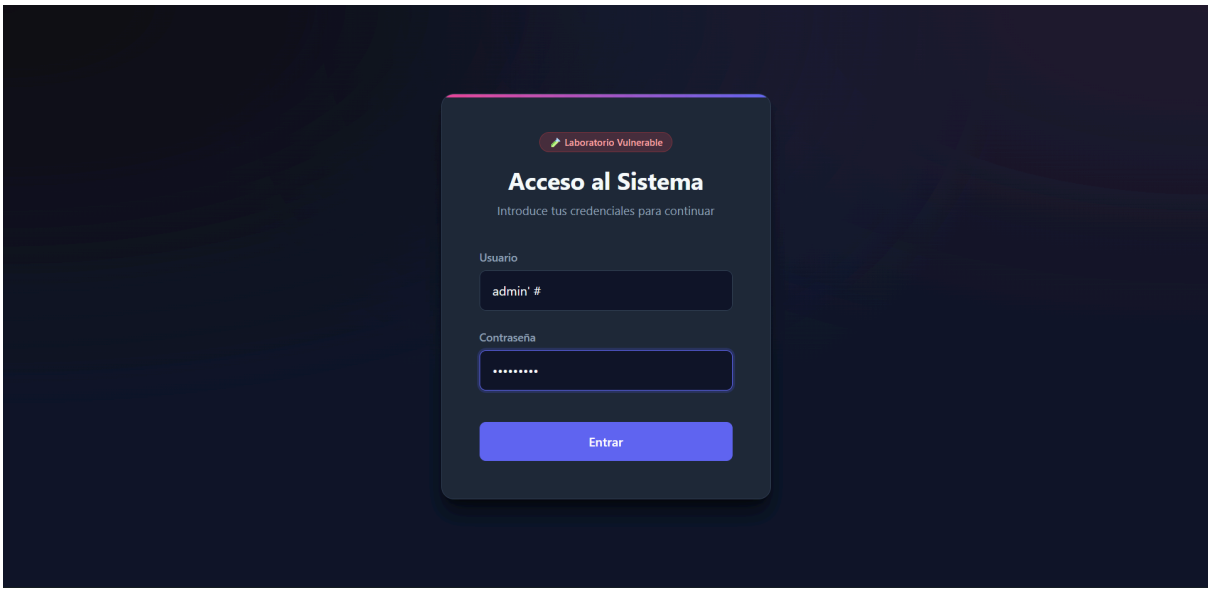
SQL Injection en PHP

Ejercicio 1 · Laboratorio vulnerable



En este escenario, se aprovecha la falta de saneamiento en el campo de usuario. Al introducir admin' #, se altera la lógica de la consulta SQL en el servidor.

- **Mecánica del ataque:** El símbolo ' cierra prematuramente la cadena del nombre de usuario, mientras que el carácter # (o -- en otros sistemas) convierte el resto de la consulta original en un comentario, ignorando la validación de la contraseña.
- **Resultado:** La SQL ejecutada se convierte en SELECT * FROM usuarios WHERE username = 'admin' #..., permitiendo el acceso como administrador sin conocer sus credenciales.

Ejercicio 2 · UNION SELECT — Volcar usuarios

Este ejercicio demuestra cómo un atacante puede extraer información que no debería ser visible, utilizando el operador UNION para combinar resultados de diferentes consultas.

- **Mecánica del ataque:** Se inyecta una sentencia que "engaña" al sistema para que devuelva filas adicionales de la tabla de usuarios.
- **Resultado:** El ataque permite visualizar en pantalla no solo al administrador, sino la lista completa de usuarios registrados, incluyendo sus contraseñas y roles.

Ejercicio 3 · Listar bases de datos

Aquí se escala el ataque para obtener información sobre la estructura del propio servidor de base de datos, apuntando a las tablas del sistema.

- **Mecánica del ataque:** Se utiliza la tabla técnica information_schema.schemata, que es un diccionario donde MySQL/MariaDB guarda los nombres de todas las bases de datos creadas.
- **Resultado:** El atacante logra listar todas las bases de datos presentes en el servidor, lo que facilita ataques posteriores más dirigidos.

Ejercicio 4 · Versión segura

- **Imposibilita Ingeniería Inversa:**
  password_hash() utiliza el algoritmo bcrypt, creando un hash unidireccional acompañado de una salt matemática aleatoria y un control de retraso (work factor). Esto impide que la base de datos contenga las contraseñas reales.

- **Mantiene privacidad ante filtraciones (Dump):**
  Si un atacante consiguiera hacer un dumpeo de la base de datos por un error en otra parte de la aplicación web, lo único que vería en la columna de contraseñas sería ruido en forma de hash (ej. $2y$10$Ua...), por lo que seguiría sin saber tus claves e imposibilitaría el uso de tablas rainbow (fuerza bruta precalculada) para desencriptarlas.

- **Validación en Backend Seguro:**
  password_verify() es el único que sabe comparar matemáticamente el texto que ha puesto tu intento de conexión ($password_input) con el hash encriptado de la Base de Datos. Solo dará OK si operativamente devuelven el mismo resultado, imposibilitando inyectar nada en este nivel aplicativo.